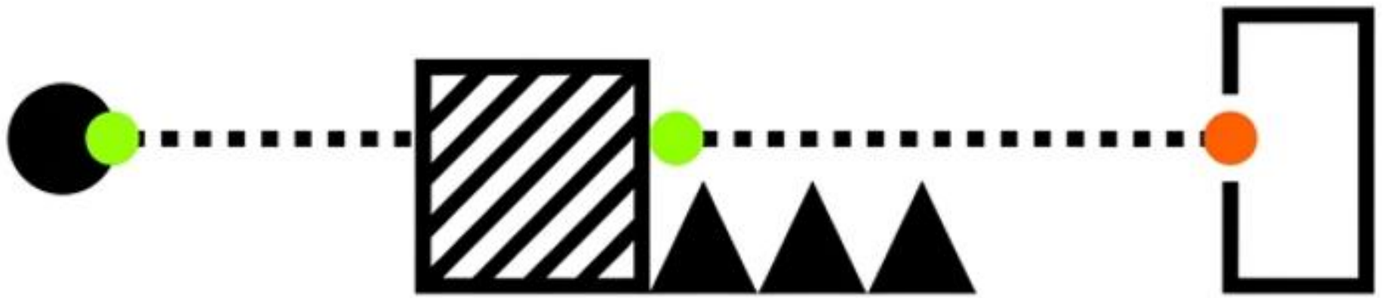
[Project: CONNECT]

게임 기획서

- 학번: 2371172
- 이름: 안태호

기획 의도 및 핵심 메커니즘 (Core Concept)



데이터 렌더링 매핑 (Data-to-Visual Mapping)

[배열 문자 (Monospace)]	[의미]	[게임 내 시각적 상징]
@	플레이어 (Player)	
#	벽 (Wall)	
.	이동 가능 통로 (Path)	
K	열쇠 (Key)	
B	장애물 (Block)	
***	함정/가시 (Trap)	
G	문/출구 (Door/Exit)	

상태 관리 대시보드 (Variables & State Management)

Player State (동적 상태)

`int playerX, playerY` 실시간 위치 좌표

`int playerHP = 5` 생명력 ♡♡♡♡♡

`int hasKey = 0` 인벤토리 상태 (0: 없음 / 1: 획득)

핵심 로직 I : 초기화 및 렌더링 (Initialize & Render)

initMap() (상태 리셋)

hasKey = 0

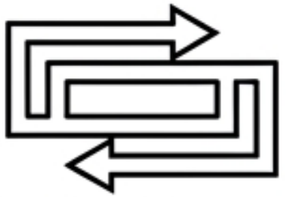
showKeyMsg = 0

playerHP = 5

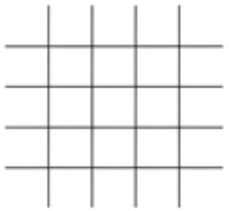
drawMap() (프레임 갱신)

`system("cls")`

화면 전체 지우기

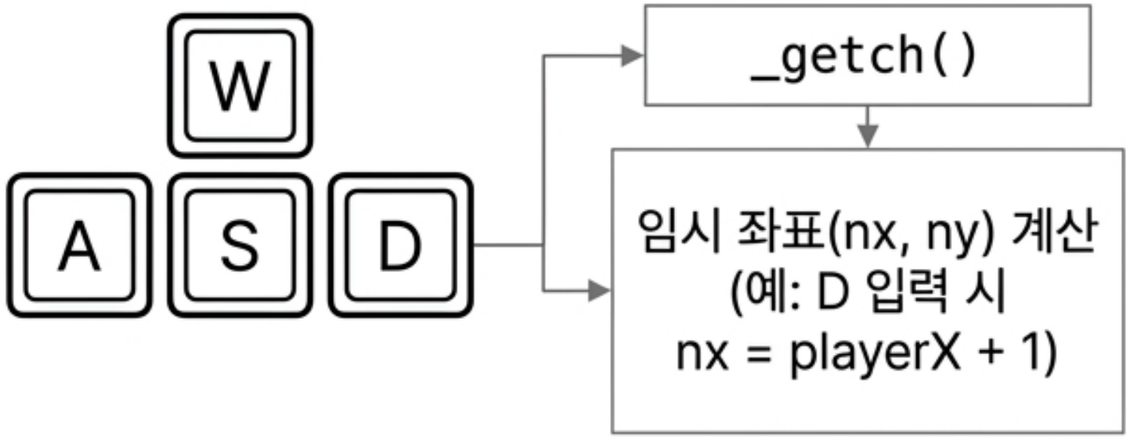


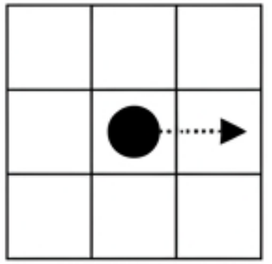
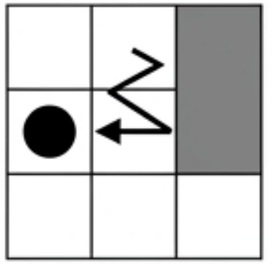
이중 for 루프 실행
(Y축, X축)



갱신된 `map[MAP_H][MAP_W]`
배열 콘솔 출력

핵심 로직 II : 이동 및 충돌 판정 (Movement & Collision)



Case 1: `map[ny][nx] == '.'`	결과: 이동 가능 -> 실제 좌표 업데이트 (playerX = nx) 
Case 2: `map[ny][nx] == '#'`	결과: 충돌/이동 불가 -> 업데이트 취소, 제자리 유지 

핵심 로직 III : 오브젝트 상호작용 (Interaction Matrix)

함정 (Trap ***)

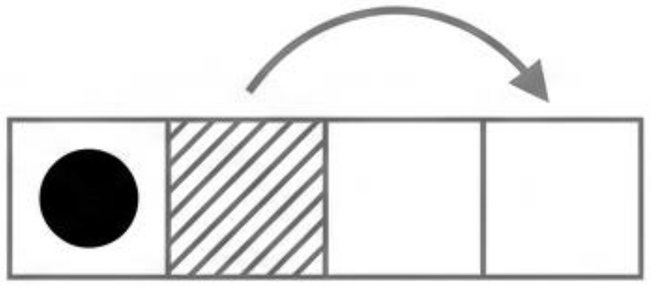
플레이어 좌표 == ***

playerHP--

장애물 밀기 (Obstacle B)

플레이어 진행 방향에 B 존재

다음 칸 ('.') 확인
-> 비어있다면 B 좌표 1칸 이동
-> 플레이어 이동



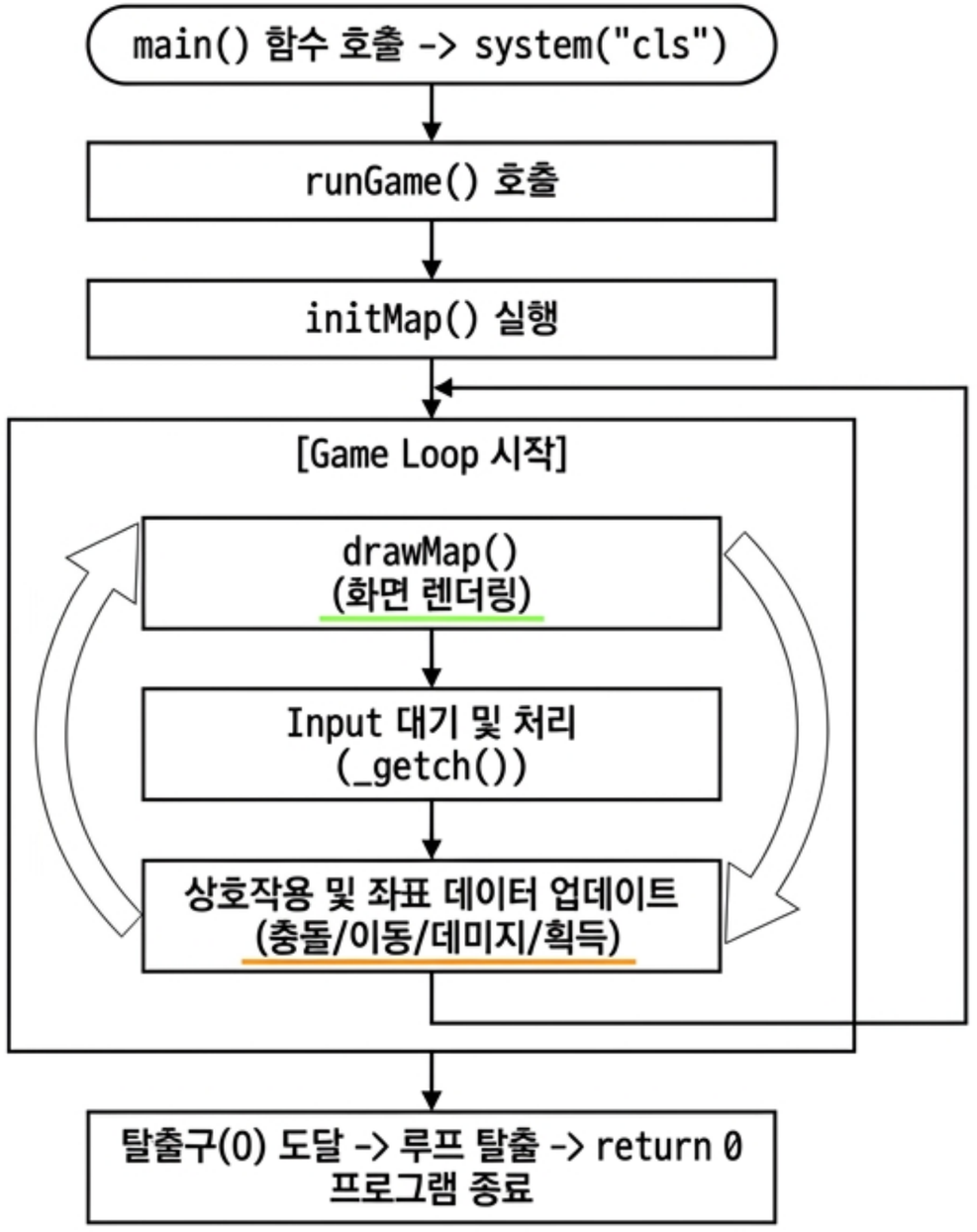
열쇠와 문 (Key K & Door O)

K와 충돌
-> hasKey = 1 상태로 변경

O와 충돌 &
hasKey == 1 검사

스테이지 Clear 조건 달성

시스템 실행 흐름도 (System Architecture & Game Loop)



아키텍처 결론 (Conclusion)

Stable Structure

C언어 콘솔 환경의 한계를 system("cls")와 이중 루프 렌더링으로 극복.

Data-Driven Design

3차원 배열(startMap)을 활용하여 코드 수정 없이 데이터만으로 무한한 스테이지 확장 가능.

Logical Scalability

명확히 분리된 상태 변수(State)와 판정 로직을 통해 추가 오브젝트(적, 새로운 함정 등) 확장에 용이한 뼈대 구축.